

```
$ onboard n8n-io/n8n --profile dev
```

n8n : un monorepo pnpm massif et très vivant, prêt pour votre première PR si vous suivez le cadre

Fair-code workflow automation platform with native AI capabilities. Combine visual building with custom code, self-host or cloud, 400+ integrations.

203 922

ÉTOILES GITHUB

10

CONTRIBUTEURS ACTIFS
sur 12 semaines

756

PR OUVERTES

10 sept. 2026

DERNIER COMMIT

<https://github.com/n8n-io/n8n> Licence Sustainable Use License Onboard · profil dev

Que fait ce projet et pour qui ?

> Une plateforme d'automatisation fair-code à 203 920 étoiles, sous Sustainable Use License : lisez LICENSE.md avant d'y bâtir un produit.

n8n est une plateforme d'automatisation de workflows « fair-code » avec des capacités IA natives : un canevas visuel combiné à du code (JavaScript, Python, packages npm), auto-hébergé ou en cloud.

Le README annonce 1500+ intégrations et 9 000+ modèles de workflows ; la description GitHub, plus ancienne, dit 400+ intégrations.

Public : les équipes qui automatisent et déploient des agents IA (OpenAI, Anthropic, Google ou modèles ouverts, sans lock-in), et les développeurs qui étendent la plateforme avec leurs propres nœuds.

Relevé le 2026-09-10 : 203 920 étoiles, 60 628 forks, 1 131 tickets ouverts (issues et PR confondues), dépôt créé le 2019-06-22, porté par l'organisation n8n-io, non archivé, branche par défaut `master` .

Licence relevée le 2026-09-10 : Sustainable Use License, version 1.0, usage et modification limités à vos besoins internes, non commerciaux ou personnels, redistribution gratuite seulement ; les fichiers `.ee.` et les branches autres que `master` en sont exclus et relèvent de la n8n Enterprise License, ce que le README résume en « fair-code »^{1,2}.

Pour voir le produit avant de toucher au code, le README propose un lancement Docker, éditeur sur <http://localhost:5678> :

```
docker volume create n8n_data
docker run -it --rm --name n8n -p 5678:5678 -v n8n_data:/home/node/.n8n docker.n8n.io/n8nio/n8n
```

Comment le code est-il organisé ?

> Tout est dans `packages/` : `cli` et `frontend/editor-ui` sont vos terrains de jeu, `core` est chasse gardée sans accord préalable.

- └ `.agents/` règles de revue pour agents IA
- └ `.claude/` config Claude Code
- └ `.devcontainer/` environnement de dev conteneurisé
- └ `.githubhooks/` hooks git
- └ `.github/` workflows CI, actions, scripts
- └ `.opencode/` config agent OpenCode
- └ `.vscode/` réglages VS Code
- └ `assets/` images et ressources
- └ `docker/` images Docker
- └ `docs/` documentation et schémas générés
- └ `packages/` code source, monorepo pnpm
- └ `patches/` patches de dépendances pnpm
- └ `scripts/` scripts de build et de dev
- └ `security/` outillage sécurité
- └ `.actrc` config act, Actions en local
- └ `.aikido` config scanner sécurité Aikido

... et 37 autres entrées à la racine du dépôt

Un monorepo pnpm workspaces piloté par Turborepo : 53 entrées à la racine, dont `packages/` (le code), `docker/` (images), `scripts/` (build et dev), `docs/` (schéma de base de données), `patches/` (patches pnpm) et `security/`.

Le `package.json` racine ne déclare que 28 devDependencies et aucune dépendance runtime : chaque package porte les siennes (`pnpm-workspace.yaml` , `pnpm-lock.yaml`).

Les packages, selon le guide de contribution et l'arborescence :

PACKAGE	RÔLE
<code>packages/cli</code>	backend, API et binaire <code>n8n</code> ; sert aussi le front
<code>packages/core</code>	moteur d'exécution des workflows et webhooks : contacter n8n avant tout changement
<code>packages/workflow</code>	interfaces partagées front et back
<code>packages/nodes-base</code>	les nœuds intégrés
<code>packages/frontend/editor-ui</code>	éditeur Vue ; <code>packages/frontend/@n8n/design-system</code> pour les composants
<code>packages/node-dev</code>	CLI pour créer un nœud
<code>packages/testing</code>	tests end-to-end Playwright
<code>packages/@n8n/</code> , <code>packages/modules</code> , <code>packages/extensions</code>	présents dans l'arborescence, rôle non relevé

Outillage à la racine : `turbo.json` (pipeline), `vitest.workspace.ts` (tests), `biome.jsonc` et `.prettierrc.js` (format), `lefthook.yml` et `.githubhooks/` (hooks), `OWNERS` (propriétaires par zone), `.devcontainer/`.

Le dépôt est écrit pour les agents IA autant que pour les humains : `AGENTS.md` , `CLAUDE.md` (qui ne fait qu'inclure `AGENTS.md`), `.github/CLAUDE.md` , `.claude/` , plus `.agents/` et `.opencode/` ; 3 repères sur 7 reconnus.

Par où commencer à lire ?

> Lisez `CONTRIBUTING.md` et `AGENTS.md` avant la première ligne de code : ils fixent l'outillage, la langue des PR et où vous n'avez pas le droit d'aller seul.

Parcours conseillé, du plus court au plus profond :

1. `README.md` puis `CONTRIBUTING.md` : structure des packages, setup, cycle de dev, règles des PR communautaires, CLA.
2. `AGENTS.md` : les règles maison (toujours pnpm, anglais technique simplifié ASD-STE100, tickets dans Linear, feature flags Posthog, jamais de secret sur la ligne de commande, travail v3 sur la branche `3.x` derrière un flag).
3. `packages/cli/bin/n8n` : le binaire visé par `pnpm start`, `webhook` et `worker` ; il vérifie la version de Node, charge la config puis le registre de commandes.
4. `packages/cli/src` : le backend, cible de `pnpm dev:be` ; `packages/cli/AGENTS.md` et `packages/cli/BREAKING-CHANGES.md` sont à côté.
5. `packages/workflow` puis `packages/core` : les interfaces partagées, puis le moteur d'exécution (`n8n-core`).
6. `packages/nodes-base` : les nœuds intégrés, le modèle à copier pour en écrire un.
7. `packages/frontend/editor-ui` : l'éditeur Vue, cible de `pnpm dev:fe:editor`.

Pour la base de données, `docs/db.md` et les schémas générés dans `docs/generated/` (tbls, SQLite et Postgres).

Comment builder, tester et lancer ?

```
> pnpm install && pnpm build && pnpm start , puis pnpm dev:be et pnpm dev:fe:editor ; la CI rejoue build, tests, typecheck, lint et e2e sur chaque PR.
```

Prérequis : Node.js 24 ou plus et pnpm 12.3.4 exactement (`packageManager` du `package.json` racine) ; un script `preinstall` bloque `npm install` .

Installation, build et lancement en mode production local, depuis votre fork :

```
npm i -g pnpm@12.3.4
git clone https://github.com/<votre_login>/n8n.git && cd n8n
git remote add upstream https://github.com/n8n-io/n8n.git
pnpm install
pnpm build
pnpm start
```

Boucle de dev : `pnpm dev:be` (backend, port 5678) et `pnpm dev:fe:editor` (éditeur, port 8080) dans deux terminaux ; `pnpm dev` à la racine n'existe plus, il affiche un avis et sort ; `pnpm dev:ai` pour les nœuds LangChain ; `N8N_DEV_RELOAD=true pnpm dev:be` recharge les nœuds à chaud.

Tests : `pnpm test` lance `turbo run test` dans tous les packages ; les tests vivent dans chaque package (Vitest : `packages/cli/test/` et six `vitest.config.*.ts` unit, integration, migration, testcontainers ; e2e Playwright dans `packages/testing` , `pnpm dev:e2e`) ; le nombre de fichiers de test n'a pas été relevé.

Qualité avant de pousser : `pnpm lint` , `pnpm typecheck` , `pnpm format` ; les hooks git lancent `actionlint` si vous touchez `.github/workflows/*.yaml` et `tbls` si vous touchez une migration.

CI : `.github/workflows/ci-pull-requests.yaml` sur `pull_request` et `merge_group` enchaîne install et build, tests unitaires, typecheck, lint, packaging, e2e Playwright, tests BDD SQLite et Postgres, sécurité ; le job `required-checks` bloque le merge.

Les autres workflows (`release-*` , `sec-*` , `test-*-nightly` , `ci-check-pr-title.yaml` , `ci-cla-check.yaml`) sont listés dans `.github/workflows/` .

Comment le projet vit-il ?

> 251 puis 249 commits par semaine, 167 PR fusionnées en quatre jours, trois lignes de release en parallèle : rebasez souvent, votre branche vieillit vite.



Dernier commit relevé le 2026-09-10 à 13:00 UTC, dernier push à 13:17 UTC le même jour.

Les deux dernières semaines relevées comptent 251 puis 249 commits ; les dix semaines précédentes apparaissent à 0, ce qui ne colle pas avec des releases quotidiennes : lisez ces zéros comme une limite du relevé, pas comme une pause, et vérifiez sur <https://github.com/n8n-io/n8n/graphs/commit-activity>.

Sur ces semaines, le premier contributeur (bjorger) signe 24 commits et il faut 16 personnes pour atteindre la moitié des commits : l'activité est répartie, pas de dépendance à une seule personne.

Releases : 10 tags relevés entre le 7 et le 10 septembre 2026, trois lignes maintenues en parallèle (`n8n@2.39.2` , `n8n@2.38.6` et `n8n@1.123.79` publiés le 10 septembre) plus les pointeurs `stable` et `beta` ; une branche `3.x` se prépare (« Resolve master→3.x sync conflict », « 3.x comparison PR »).

Flux : 167 PR fusionnées relevées entre le 7 et le 10 septembre (la fenêtre relevée fait quatre jours, pas trente), 756 PR ouvertes, 375 issues ouvertes, 300 fermées en 30 jours.

Le CHANGELOG n'a pas été relevé (fichier trop volumineux) : <https://github.com/n8n-io/n8n/blob/master/CHANGELOG.md> et <https://github.com/n8n-io/n8n/releases> font foi.

Sécurité et gouvernance : `SECURITY.md` renvoie au programme de divulgation de vulnérabilités, un fichier `OWNERS` attribue les zones, pas de fichier de financement.

Quelle première contribution ?

> Pas de « good first issue » relevée : choisissez un nœud ou un bug d'éditeur reproductible, restez hors de `packages/core`, signez le CLA et respectez le titre `type(scope): sujet`.

Aucune issue « good first issue » relevée le 2026-09-10 : la liste est vide ; cherchez sur <https://github.com/n8n-io/n8n/issues>.

Les labels sont surtout des marqueurs d'équipe et de statut (status:in-linear 297, status:team-assigned 280, team:cats 93, team:nodes 92, team:ai 61, Needs Feedback 41) : le triage se fait dans Linear, hors GitHub.

Les issues les plus discutées sont déjà assignées à une équipe : #31837 « Webhook trigger firing multiple times » (17 commentaires), #29127 objet imbriqué muté entre branches sœurs (15) ; bonnes lectures pour comprendre le moteur, mauvais premiers tickets.

La voie du guide de contribution : fork, remote `upstream`, tests, PR depuis le fork, CLA signé ; les PR communautaires ont leurs règles (Community PR Guidelines) et deux workflows dédiés existent, `community-pr-close-stale.yml` et `ci-cla-check.yml` : une PR inactive risque la fermeture, le CLA est vérifié ; le titre est contrôlé (`ci-check-pr-title.yml`) au format `type(scope): sujet`, par exemple `fix(AI Agent Node): ...`.

VOUS VOULEZ	OÙ	GARDE-FOU
Corriger ou ajouter un nœud	<code>packages/nodes-base</code> , section « Create custom nodes » du guide	<code>pnpm dev</code> dans <code>packages/nodes-base</code> + <code>N8N_DEV_RELOAD=true</code> <code>pnpm dev</code> dans <code>packages/cli</code>
Un bug d'éditeur	<code>packages/frontend/editor-ui</code>	<code>pnpm dev:fe:editor</code> , tests Vitest du package
Une commande ou une route API	<code>packages/cli/src</code>	<code>pnpm dev:be</code> , tests d'intégration dans <code>packages/cli/test/</code>
Le moteur d'exécution	<code>packages/core</code>	contacter n8n avant de commencer

Tableau fondé sur le guide de contribution et l'arborescence ; les PR s'écrivent en anglais technique simplifié, phrases courtes, voix active.

\$ Vos 3 premières actions

1. Installer et lancer en local, puis ouvrir <http://localhost:5678> : `npm i -g pnpm@12.3.4` , `pnpm install` , `pnpm build` , `pnpm start` ; Node 24 minimum, jamais npm dans le dépôt.
2. Lire `CONTRIBUTING.md` , `AGENTS.md` et `packages/cli/AGENTS.md` , puis lancer la boucle de dev `pnpm dev:be` + `pnpm dev:fe:editor` et faire tourner `pnpm test` une fois pour connaître la durée de référence.
3. Choisir un premier ticket sur <https://github.com/n8n-io/n8n/issues> (aucune « good first issue » relevée le 2026-09-10), de préférence un nœud dans `packages/nodes-base` ou un bug d'éditeur, signer le CLA, ouvrir la PR depuis votre fork avec un titre `type(scope): sujet` .

Sources

1. <https://github.com/n8n-io/n8n/blob/master/LICENSE.md>
2. https://github.com/n8n-io/n8n/blob/master/LICENSE_EE.md